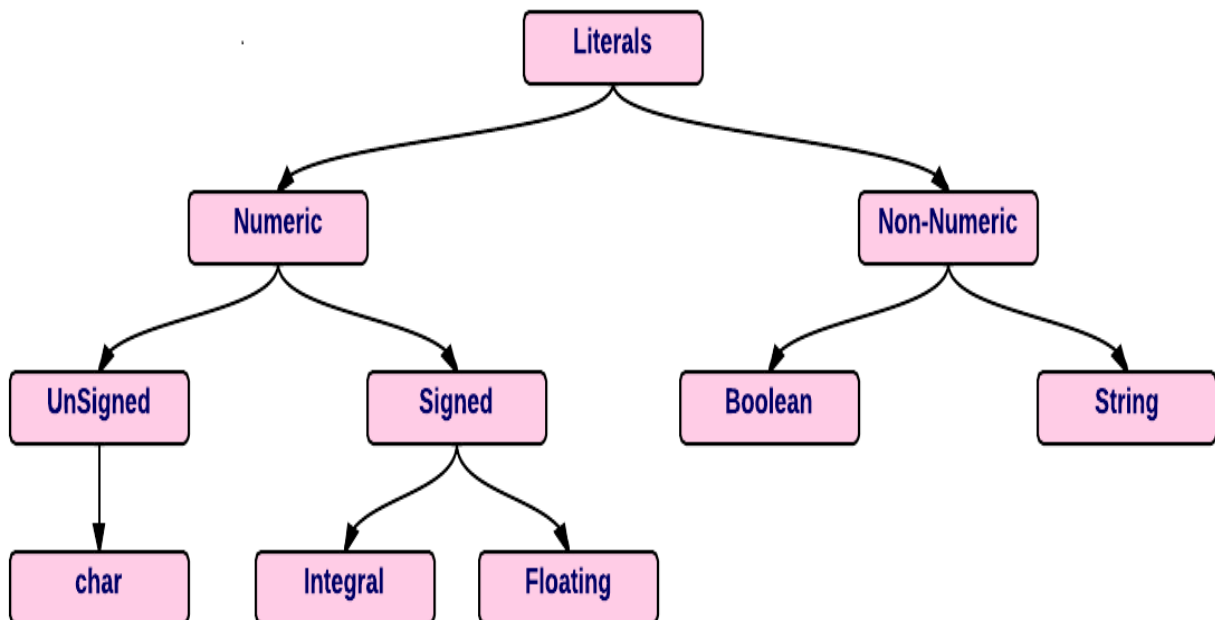
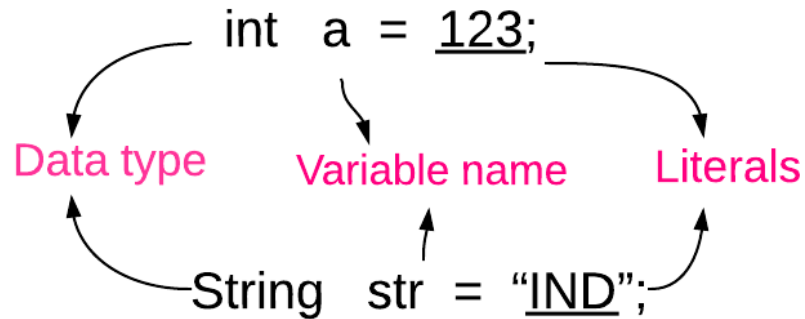


## 7. Literals

- ❖ Literals are the actual value. It is used to perform any operation.
- ❖ Constant values in program are called as literals.

Example:



### Types of Literals

1. Boolean Literal
2. Character Literal
3. String Literal
4. Integral Literal
5. Floating Literal
6. null Literal

## 1. Boolean Literal

- ❖ Boolean Literal can be assigned to a boolean type variable.
- ❖ There are two boolean literals.
  - true
  - false
- ❖ A boolean type variable can hold value as true or false.
- ❖ A boolean type variable cannot hold numeric value.
- ❖ Boolean literals – it is reserved words and must be in lower case.

### Program:

#### JAVA31.java

```
class JAVA31{
public static void main (String args []){
boolean b1 = true;
boolean b2 = false;
System.out.println(b1);
System.out.println(b2);
}
}
```

#### JAVA32.java

```
class JAVA32{
public static void main (String args []){
boolean b1 = 0;
System.out.println(b1);
}
}
```

#### JAVA33.java

```
class JAVA33{
public static void main (String args []){
boolean b1 = True;
System.out.println(b1);
}
}
```

#### JAVA34.java

```
class JAVA34{
public static void main (String args []){
boolean True=false;
boolean b=True;
System.out.println(b);
}
}
```

## 2. Character Literal

- ❖ Character Literal is a single character enclosed within single quotation marks.
- ❖ Space and Tab is valid character so it can be placed in single quotation marks.
- ❖ A character type variable can hold following
  - ⇒ Single character enclosed within single quotation marks.
  - ⇒ Escape Sequence
  - ⇒ ASCII value
  - ⇒ UNICODE Character.
  - ⇒ Octal character

### Program:

#### JAVA35.java

```
class JAVA35{
public static void main (String args []){
char ch1='a';
char ch2='J';
char ch3='5';
char ch4='*';
}
```

```
System.out.println(ch1);
System.out.println(ch2);
System.out.println(ch3);
System.out.println(ch4);
}
}
```

**JAVA36.java**

```
class JAVA36{
public static void main (String args []){
char ch1=""; // empty
System.out.println(ch1);
}
}
```

**JAVA37.java**

```
class JAVA37{
public static void main (String args []){
char ch1=' '; // One space
System.out.println(ch1);
}
}
```

**JAVA38.java**

```
class JAVA38{
public static void main (String args []){
char ch1='  '; // Two space
System.out.println(ch1);
}
}
```

**JAVA39.java**

```
class JAVA39{
public static void main (String args []){
char ch1='  '; // One Tab
System.out.println(ch1);
}
}
```

**JAVA40.java**

```
class JAVA40{
public static void main (String args []){
char ch1='  '; // Two Tab
System.out.println(ch1);
}
}
```

**JAVA41.java**

```
class JAVA41{
public static void main (String args []){
char ch1='\\';
System.out.println(ch1);
}
}
```

**JAVA42.java**

```
class JAVA42{
public static void main (String args []){
char ch1='+';
char ch2='\"';
System.out.println(ch1);
System.out.println(ch2);
}
}
```

**I. Escape Sequence**

- ❖ Escape Sequence is a special notation which is used to request some special characters which can't be represented as it is.

| Escape Sequence | Description            |
|-----------------|------------------------|
| \t              | Tab Space              |
| \b              | Back Space             |
| \n              | Newline                |
| \r              | Carriage return        |
| \f              | Formfeed               |
| \'              | Single Quote character |
| \"              | Double Quote character |
| \\              | Backslash character    |

- ❖ Backspace character (\b) is used to form escape sequence so only backspace character can't be placed in single quotation marks.
- ❖ Single Quote is used to form character literal so only single quote can't be placed in single quotation marks.

- ❖ If you want to represent that character then you have to represent as Escape Sequence.
- ❖ Every character enclosed in single quotations marks will have an integer equivalent value called as ASCII Value.
- ❖ char type variable can hold integer value ranging from 0-65535 only.
- ❖ char type variable can hold UNICODE value ranging from \u0000-\uFFFF only.
- ❖ char type variable can hold Octal value ranging from \0-\377 only.
- ❖ When you print the char type variable which holds Integer, Unicode or Octal value then it display ? which indicates no character defined for that value.
- ❖ When you assign character literals to int type variables then internally ASCII value of that character will be assigned.

**Program:**

|                                                                                                                                                           |                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JAVA43.java</b><br><pre> class JAVA43{ public static void main (String args []){ char ch1="\\"; System.out.println(ch1); } } </pre>                    | <b>JAVA44.java</b><br><pre> class JAVA44{ public static void main (String args []){ char ch1="\\"; System.out.println(ch1); } } </pre>                   |
| <b>JAVA45.java</b><br><pre> class JAVA45{ public static void main (String args []){ char ch1='\+'; System.out.println(ch1); } } </pre>                    | <b>JAVA46.java</b><br><pre> class JAVA46{ public static void main (String args []){ char ch1='\p'; System.out.println(ch1); } } </pre>                   |
| <b>JAVA47.java</b><br><pre> class JAVA47{ public static void main (String args []){ char ch1='\a'; System.out.println(ch1); } } </pre>                    | <b>JAVA48.java</b><br><pre> class JAVA48{ public static void main (String args []){ char ch1='\"'; System.out.println(ch1); } } </pre>                   |
| <b>JAVA49.java</b><br><pre> class JAVA49{ public static void main (String args []){ char ch1='\"'; System.out.println(ch1); } } </pre>                    | <b>JAVA50.java</b><br><pre> class JAVA50{ public static void main (String args []){ String str="welcome"\"kamlash\"; System.out.println(str); } } </pre> |
| <b>JAVA51.java</b><br><pre> class JAVA51{ public static void main (String args []){ String str="Welcome to \" ABC\""; System.out.println(str); } } </pre> | <b>JAVA52.java</b><br><pre> class JAVA52{ public static void main (String args []){ char ch1='\u'; System.out.println(ch1); } } </pre>                   |

## II. ASCII Character Sets

- ❖ ASCII → American Standard Code for Information Interchange.
- ❖ Every character enclosed in single quotation marks will have an integer equivalent value called as ASCII value.
- ❖ ASCII Value Range is 0-255.
- ❖ ASCII Value can be assigned to a char type variable.

### Program:

|                                                                                                                                                                                                                |                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JAVA53.java</b><br><pre>class JAVA53{ public static void main (String args []){ char ch='1'; System.out.println(ch); } }</pre>                                                                              | <b>JAVA54.java</b><br><pre>class JAVA54{ public static void main (String args []){ char ch=1; System.out.println(ch); } }</pre>                                            |
| <b>JAVA55.java</b><br><pre>class JAVA55{ public static void main (String args []){ char ch='1'; System.out.println(ch); System.out.println(ch+10); } }</pre>                                                   | <b>JAVA56.java</b><br><pre>class JAVA56{ public static void main (String args []){ System.out.println(1+2); System.out.println('1'+'2'); } }</pre>                         |
| <b>JAVA57.java</b><br><pre>class JAVA57{ public static void main (String args []){ char ch1=65; char ch2=48; char ch3=97; System.out.println(ch1); System.out.println(ch2); System.out.println(ch3); } }</pre> | <b>JAVA58.java</b><br><pre>class JAVA58{ public static void main (String args []){ char ch1=260; char ch2='?'; System.out.println(ch1); System.out.println(ch2); } }</pre> |
| <b>JAVA59.java</b><br><pre>class JAVA59{ public static void main (String args []){ char ch1=0; char ch2=65535; System.out.println(ch1); System.out.println(ch2); } }</pre>                                     | <b>JAVA60.java</b><br><pre>class JAVA60{ public static void main (String args []){ char ch1=1; char ch2=65536; System.out.println(ch1); System.out.println(ch2); } }</pre> |

**JAVA61.java**

```
class JAVA61{
public static void main (String args []){
char ch1=253;
System.out.print("(a+b)");
System.out.print(ch1);
System.out.println();
}
```

```
char ch2=246;
System.out.print("10");
System.out.print(ch2);
System.out.print("2=5");
}
}
```

**III. UNICODE Characters**

- ❖ UNICODE → UNiversal CODE.
- ❖ Every character will have UNICODE VALUE.
- ❖ UNICODE Notation:

Syntax:

\uXXXX      Here, X is Hexadecimal digit.  
Starts with \u followed by four hexadecimal digits.

UNICODE Range:

\u0000(0) to \uFFFF

- ❖ Source code is also written in UNICODE representation.

Example:

|                       |   |            |         |
|-----------------------|---|------------|---------|
| int a=\u0037;         | → | int a =7;  | Valid   |
| int a='\u0037'        | → | int a='7'; | Valid   |
| int a=\u0037 \u0037   | → | int a=79;  | Valid   |
| int a='\u0037 \u0037' | → | int a='79' | Invalid |

**Program:**

**JAVA62.java**

```
class JAVA62{
public static void main (String args []){
char a='\u0061';
char b='\u0062';
System.out.print(a);
System.out.println(b);
char c='\u0041';
char d='\u0042';
System.out.print(c);
System.out.println(d);
}
}
```

**JAVA63.java**

```
class JAVA63{
public static void main (String args []){
int a='\u0061';
int b='\u0062';
System.out.println(a);
System.out.println(b);
int c='\u0041';
int d='\u0042';
System.out.println(c);
System.out.println(d);
}
}
```

**JAVA64.java**

```
class JAVA64{
public static void main (String args []){
int a=\u0037;
int b=\u0039;
System.out.println(a);
System.out.println(b);
}}
}
```

**JAVA65.java**

```
class JAVA65{
public static void main (String args []){
int a=\u0037\u0039;
System.out.println(a);
}
}
```

**JAVA66.java**

```
class JAVA66{
public static void main (String args []){
int a='\u0037\u0039';
System.out.println(a);
}
}
```

**JAVA67.java**

```
class JAVA67{
public static void main (String args []){
int \u0061=99;
System.out.println(a);
}
}
```

**JAVA68.java**

```
class JAVA68{
public static void main (String args []){
int a=\u0061;
System.out.println(a);
}
}
```

**JAVA69.java**

```
class JAVA69{
public static void main (String args []){
int a=99;
int x=\u0061;
System.out.println(x);
}
}
```

**JAVA70.java**

```
class JAVA70{
pu \u0062li\u0063 st\u0061ti\u0063 void m\u0061in (String arg[]){
int \u0041\u0042=123;
System.out.println(AB);
}
}
```

**IV. Octal value as Char type**

- ❖ Octal value can be assigned to char type.
- ❖ Octal Notation:

Syntax:

\DDD                      Here, D is octal digit.

Starts with \ followed by three octal digits.

UNICODE Range:

Range in Decimal    0   -   255

Range in Octal        \O   -   \377

**Table of ASCII, OCTAL, UNICODE for Numerical value**

| Character | ASCII | OCTAL | UNICODE | Character | ASCII | OCTAL | UNICODE |
|-----------|-------|-------|---------|-----------|-------|-------|---------|
| 0         | 48    | 060   | \u0030  | 5         | 53    | 065   | \u0035  |
| 1         | 49    | 061   | \u0031  | 6         | 54    | 066   | \u0036  |
| 2         | 50    | 062   | \u0032  | 7         | 55    | 067   | \u0037  |
| 3         | 51    | 063   | \u0033  | 8         | 56    | 070   | \u0038  |
| 4         | 52    | 064   | \u0034  | 9         | 57    | 071   | \u0039  |

Table of ASCII, OCTAL, UNICODE for Alphabet

| Character | ASCII | OCTAL | UNICODE | Character | ASCII | OCTAL | UNICODE |
|-----------|-------|-------|---------|-----------|-------|-------|---------|
| A         | 65    | 101   | \u0041  | a         | 97    | 141   | \u0061  |
| B         | 66    | 102   | \u0042  | b         | 98    | 142   | \u0062  |
| C         | 67    | 103   | \u0043  | c         | 99    | 143   | \u0063  |
| D         | 68    | 104   | \u0044  | d         | 100   | 144   | \u0064  |
| E         | 69    | 105   | \u0045  | e         | 101   | 145   | \u0065  |
| F         | 70    | 106   | \u0046  | f         | 102   | 146   | \u0066  |
| G         | 71    | 107   | \u0047  | g         | 103   | 147   | \u0067  |
| H         | 72    | 108   | \u0048  | h         | 104   | 150   | \u0068  |
| I         | 73    | 109   | \u0049  | i         | 105   | 151   | \u0069  |
| J         | 74    | 110   | \u004A  | j         | 106   | 152   | \u006A  |
| K         | 75    | 111   | \u004B  | k         | 107   | 153   | \u006B  |
| L         | 76    | 112   | \u004C  | l         | 108   | 154   | \u006C  |
| M         | 77    | 113   | \u004D  | m         | 109   | 155   | \u006D  |
| N         | 78    | 114   | \u004E  | n         | 110   | 156   | \u006E  |
| O         | 79    | 115   | \u004F  | o         | 111   | 157   | \u006F  |
| P         | 80    | 116   | \u0050  | p         | 112   | 160   | \u0070  |
| Q         | 81    | 117   | \u0051  | q         | 113   | 161   | \u0071  |
| R         | 82    | 118   | \u0052  | r         | 114   | 162   | \u0072  |
| S         | 83    | 119   | \u0053  | s         | 115   | 163   | \u0073  |
| T         | 84    | 120   | \u0054  | t         | 116   | 164   | \u0074  |
| U         | 85    | 121   | \u0055  | u         | 117   | 165   | \u0075  |
| V         | 86    | 122   | \u0056  | v         | 118   | 166   | \u0076  |
| W         | 87    | 123   | \u0057  | w         | 119   | 167   | \u0077  |
| X         | 88    | 124   | \u0058  | x         | 120   | 170   | \u0078  |
| Y         | 89    | 125   | \u0059  | y         | 121   | 171   | \u0079  |
| Z         | 90    | 126   | \u005A  | z         | 122   | 172   | \u007A  |

**Program:****JAVA71.java**

```

class JAVA71{
public static void main (String args []){
char ch1='\101';
char ch2='\103';
System.out.println(ch1);
System.out.println(ch2);
}}

```

**JAVA72.java**

```

class JAVA72{
public static void main (String args []){
char ch='\101';
System.out.println(ch);
}
}

```

**JAVA73.java**

```

class JAVA73{
public static void main (String args []){
int a='\101';
System.out.println(a);
}
}

```

**JAVA74.java**

```

class JAVA74{
public static void main (String args []){
int a=\101;
System.out.println(a);
}
}

```



**JAVA75.java**

```

class JAVA75{
public static void main (String args []){
char ch='\402';
System.out.println(ch);
}
}

```

**3. String Literal**

- ❖ String Literal is a collection of zero or more characters enclosed between double quotation marks.
- ❖ String Literals can be assigned to reference variable of type String.
- ❖ String Literal will be represented as char array in memory.
- ❖ Empty String Literals is allowed where as empty character literals is not allowed.
- ❖ If reference variable of String type contains null value then it is called as NULL String.
- ❖ You can't refer any members with NULL String. If you do so it result is NullPointerException.
- ❖ If Reference variable of String type contains zero character in double quotations marks then it is called as EMPTY String.
- ❖ You can use escape sequence to represent UNICODE/Octal value as it is in String literals.

**Program:****JAVA76.java**

```

class JAVA76{
public static void main (String args []){
String str =null;
System.out.println(str);
int x=str.length();
System.out.println("len: " + x);
}
}

```

**JAVA77.java**

```

class JAVA77{
public static void main (String args []){
String str ="";
System.out.println(str);
int x=str.length();
System.out.println("len: "+x);
}
}

```

**JAVA78.java**

```

class JAVA78{
public static void main (String args []){
String str ="AB12@$ab";
System.out.println(str);
int x=str.length();
System.out.println("len: "+x);
}
}

```

**JAVA79.java**

```

class JAVA79{
public static void main (String args []){
String str1 ="ASCII of A is 65";
System.out.println(str1);
String str2 ="UNICODE of A is\u0041";
System.out.println(str2);
String str3 ="UC of A is \\u0041";
System.out.println(str3);
}
}

```

**JAVA80.java**

```
class JAVA80{
public static void main (String args []){
String str ="D:\new\test\branch";
System.out.println(str);
String str1 ="D:\\new\\test\\branch";
System.out.println(str1);
}
}
```

**JAVA81.java**

```
class JAVA81{
public static void main (String args []){
String str ="D:\java\core";
System.out.println(str);
}
}
```

**JAVA82.java**

```
class JAVA82{
public static void main (String args []){
String str ="D:\\java\\core";
System.out.println(str);
}
}
```

**JAVA83.java**

```
class JAVA83{
public static void main (String args []){
String str ="Octal of 65 is \101";
System.out.println(str);
String str1 ="Octal of 65 is \\101";
System.out.println(str1);
}
}
```

**4. Integral Literal**

- ❖ Integer Literals can be assigned to one of the integral data type- byte, short, int, long.
- ❖ Integer Literals size is depending on integral data type you are using.
- ❖ Default type of Integer Literal is int.
- ❖ Integer value range from -2147483648 to 2147483648.
- ❖ If you want to represent the integer value out of range then you need to use L/l suffix.

There are four types of Integer Literals.

- I. Decimal Literals
- II. Octal Literals
- III. Hexadecimal Literals
- IV. Binary Literals (java 7)

**I. Decimal Literals**

- ❖ Decimal Literal is a valid number from decimal number system.
- ❖ The base or radix of decimal number System is 10.  
i.e. only 10 digits ranging from 0-9 are allowed to form the decimal literals.
- ❖ Decimal Literals cannot start with 0.

**Program:****JAVA84.java**

```
class JAVA84{
public static void main (String args []){
byte b = 65;
System.out.println(b);
}
```

```
short s = 1234;
System.out.println(s);
int i= 123431;
System.out.println(i);
}
}
```

**JAVA85.java**

```
class JAVA85{  
public static void main (String args []){  
byte b = 128;  
System.out.println(b);  
}  
}
```

**JAVA86.java**

```
class JAVA86{  
public static void main (String args []){  
short s = 32768;  
System.out.println(s);  
}  
}
```

**JAVA87.java**

```
class JAVA87{  
public static void main (String args []){  
int i= 2147483648;  
System.out.println(i);  
}  
}
```

**JAVA88.java**

```
class JAVA88{  
public static void main (String args []){  
Long L= 2147483648;  
System.out.println(L);  
}  
}
```

**JAVA89.java**

```
class JAVA89{  
public static void main (String args []){  
Long L= 2147483648L;  
System.out.println(L);  
}  
}
```

**JAVA90.java**

```
class JAVA90{  
public static void main (String args []){  
System.out.println(9980979999);  
}  
}
```

**JAVA91.java**

```
class JAVA91{  
public static void main (String args []){  
System.out.println(9980979999L);  
}  
}
```

**JAVA92.java**

```
class JAVA92{  
public static void main (String args []){  
int a=098;  
System.out.println(a);  
}  
}
```

## II. Octal Literals

- ❖ Octal Literal is a valid number from octal number system.
- ❖ The base or radix of octal number System is 8.  
i.e. only 8 digits ranging from 0-7 are allowed to form the Octal literals.
- ❖ Octal Literals cannot start with 0.
- ❖ When java compiler encounters Octal, Binary or Hexadecimal literals then that will be replaced with decimal equivalent value.

### Program:

**JAVA93.java**

```
class JAVA93{  
public static void main (String args []){  
int a=1010;  
int b=0101;  
System.out.println(a);  
System.out.println(b);  
}  
}
```

### III. Hexadecimal Literals

- ❖ Hexadecimal Literal is a valid number from hexadecimal number system.
- ❖ The base or radix of hexadecimal number System is 16.  
i.e. only 16 digits ranging from 0-9 and A/a to F/f are allowed to form the Hexadecimal literals
- ❖ Hexadecimal Literals must start with 0X /0x.  
Example: 0x 123af
- ❖ When java compiler encounters Octal, Binary or Hexadecimal literals then that will be replaced with decimal equivalent value.

#### Program:

##### JAVA94.java

```
class JAVA94{
public static void main (String args []){
int a=0XA;
int b=0x61;
int c=0xface;
System.out.println(a);
System.out.println(b);
System.out.println(c);
}
}
```

##### JAVA95.java

```
class JAVA95{
public static void main (String args []){
int c=0xFACE;
System.out.println(c);
}
}
```

### IV. Binary Literals

- ❖ Binary Literal is a valid number from binary number system.
- ❖ The base or radix of Binary number System is 2.  
i.e. only 2 digits 0 and 1 are allowed to form the Binary literals.
- ❖ Binary Literals must start with 0B /0b.
- ❖ When java compiler encounters Octal, Binary or Hexadecimal literals then that will be replaced with decimal equivalent value.

#### Program:

##### JAVA96.java

```
class JAVA96{
public static void main (String args []){
int a=1010;
int b=0b101;
System.out.println(a);
System.out.println(b);
}}
}
```

##### JAVA97.java

```
class JAVA97{
public static void main (String args []){
int a=0b102;
System.out.println(a);
}
}
```

##### JAVA98.java

```
class JAVA98{
public static void main (String args []){
int a=123;
int b=0101;
int c=0x101;
int d=0B101;
}
```

```
System.out.println(a);
System.out.println(b);
System.out.println(c);
System.out.println(d);
}
}
```

## 5. Floating Literal

- ❖ Floating point literals can be assigned to one of the floating point data type – float, double.
- ❖ Floating point literals size is depending on floating point type you are using.
- ❖ Default type of floating Point literals is double.
- ❖ You can use D/d as a suffix for the double value optionally.
- ❖ You must use F/f as a suffix for float value.
- ❖ When floating point value starts with 0 then it will be considered as Floating Point Literals only, not as Octal Literal.
- ❖ When Integer value starts with 0X/0x then it will be considered as Hexadecimal Literals.
- ❖ When floating point value starts with 0X/0x then it must used P in fractional part.
- ❖ When Integer value starts with 0B/0b then it will be considered as Binary Literals.
- ❖ Floating point cannot be represented as Binary Literals.
- ❖ Floating point literals can be represented using following notations.
  - ⇒ Standard Floating point Notation.
  - ⇒ Exponent/Scientific Notation.
  - ⇒ Hexadecimal Notation (java5)

### I. Standard Floating point Notation

- ❖ It consists of a whole number followed by a decimal point and fractional part

Example: 12987.63

**Program:**

|                                                                                                                                                                                        |                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JAVA99.java</b><br><pre> class JAVA99{ public static void main (String args []){ double d1=1234.56; double d2=123.456; System.out.println(d1); System.out.println(d2); } } </pre>   | <b>JAVA100.java</b><br><pre> class JAVA100{ public static void main (String args []){ float f=12987.63; System.out.println(f); } } </pre> |
| <b>JAVA101.java</b><br><pre> class JAVA101{ public static void main (String args []){ float f1=1234.78f; float f2=12.7568f; System.out.println(f1); System.out.println(f2); } } </pre> |                                                                                                                                           |

## II. Scintific/Exponent Notation

- ❖ The Exponent is indicated by an E or e followed by a decimal number, which can be positive or negative.

$123.4567e2 \rightarrow 123.4567 \times 10^2$   
 $123.4567E+2 \rightarrow 123.4567 \times 10^2$   
 $123.4567e-2 \rightarrow 123.4567 \times 10^{-2}$

**Program:**

|                                                                                                                                                                |                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JAVA102.java</b><br><pre> class JAVA102{ public static void main (String args []){ float f=129.8763e2f; double d1=12.8763e+2; double d2=12987.63e-2; </pre> | <pre> System.out.println(f); System.out.println(d1); System.out.println(d2); } } </pre>                                                  |
| <b>JAVA103.java</b><br><pre> class JAVA103{ public static void main (String args []){ double d=077; System.out.println(d); } } </pre>                          | <b>JAVA104.java</b><br><pre> class JAVA104{ public static void main (String args []){ double d=077.0; System.out.println(d); } } </pre>  |
| <b>JAVA105.java</b><br><pre> class JAVA105{ public static void main (String args []){ double d=0xA9; System.out.println(d); } } </pre>                         | <b>JAVA106.java</b><br><pre> class JAVA106{ public static void main (String args []){ double d=0xA9.0; System.out.println(d); } } </pre> |

## II. Hexadecimal Floating Point Literals

- ❖ It allowed literals of the float and double types to be written primarily in base 16 rather than base 10.

Double x=0XaP0;  
 a -> 10      //  $10 \times 2^0 = 10.0$

**Program:**

|                                                                                                                                                                                           |                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JAVA107.java</b><br><pre> class JAVA107{ public static void main (String args []){ double d=(0xA9.1p0); double d1=(0xA9.p0); System.out.println(d); System.out.println(d1); } } </pre> | <b>JAVA108.java</b><br><pre> class JAVA108{ public static void main (String args []){ double d=0xAp0; double d1=0xAp1; double d2=0xAp2; System.out.println(d); System.out.println(d1); System.out.println(d2); }} </pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
double y=0Xfp2D
    f -> 10
    //15*22=60.0
float z=0Xf.aP1F;
    f -> 15
    a -> 10
    //(15 * 160 +10 /161) * 21
    //(15 * 1 +.625) * 2
    //(15.625 * 2) = 31.25f
```

**JAVA109.java**

```
class JAVA109{
public static void main (String args []){
float d=0Xfp2D;
System.out.println(d);
}
}
```

**JAVA110.java**

```
class JAVA110{
public static void main (String args []){
float d=0xf.ap1f;
System.out.println(d);
}
}
```

```
double y=0X1f.a5P1
    //(1 * 161 +15 * 160 +10 /161 + 5/162) * 21
    //(16 +15 +10 /16+ 5/256) * 2
    //(16 +15 +.625 + .01953125) * 2
    //(31.64453125)
    // 63.2890625
```

**JAVA111.java**

```
class JAVA111{
public static void main (String args []){
float f=0x1f.a5p1;
System.out.println(f);
}
}
```

**JAVA112.java**

```
class JAVA112{
public static void main (String args []){
double d =0b1011;
System.out.println(d);
}
}
```

**JAVA113.java**

```
class JAVA113{
public static void main (String args []){
double d =0b1011.1;
System.out.println(d);
}
}
```

**6. null Literal**

- ❖ Null is a value.
- ❖ It is default value for any reference variable.
- ❖ If the value of reference variable is null then it indicates that address/reference is not available in the variable.

**UNDERSCORE in Number Literals**

- ❖ To represent unit of the values you can use comma (,) symbol in number representation.

Example:

2,45,12,123

- ❖ For same thing in java, you have to use Underscore (\_) to represent unit of the values.

Example:

2\_45\_12\_123

- ❖ When UNDERSCORE is used before first digit then it will considered as variable.
- ❖ You can't use UNDERSCORE after the last digit.
- ❖ In the case of Octal literal UNDERSCORE can be used just after the 0.
- ❖ In the case of Binary literal UNDERSCORE can be used just after the 0b/0B.
- ❖ In the case of Hexadecimal literal UNDERSCORE can be used just after the 0x/0X.
- ❖ You can use more than one UNDERSCORE side by side between two digits.
- ❖ You can use UNDERSCORE with floating point literals but not allowed just before of after the decimal point.

**Programs:****JAVA114.java**

```
class JAVA114{
public static void main (String args []){
int a=24512452;
int b=2,45,12,452;
System.out.println(a);
System.out.println(b);
}
}
```

**JAVA115.java**

```
class JAVA115{
public static void main (String args []){
int a=2_44_52_452;
System.out.println(a);
}
}
```

**JAVA116.java**

```
class JAVA116{
public static void main (String args []){
int _12=9898;
System.out.println(_12);
int a=_12;
System.out.println(a);
}
}
```

**JAVA117.java**

```
class JAVA117{
public static void main (String args []){
int _12=_123;
System.out.println(_12);
}
}
```



**JAVA118.java**

```
class JAVA118{  
public static void main (String args []){  
int a=122_  
System.out.println(a);  
}  
}
```

**JAVA119.java**

```
class JAVA119{  
public static void main (String args []){  
int a=0_77;  
System.out.println(a);  
}  
}
```

**JAVA120.java**

```
class JAVA120{  
public static void main (String args []){  
int a=0b1_01_01;  
System.out.println(a);  
}  
}
```

**JAVA121.java**

```
class JAVA121{  
public static void main (String args []){  
int a=0b_101;  
System.out.println(a);  
}  
}
```

**JAVA122.java**

```
class JAVA122{  
public static void main (String args []){  
int a=8_9_81;  
System.out.println(a);  
}  
}
```

**JAVA123.java**

```
class JAVA123{  
public static void main (String args []){  
int a=0x1_A_F;  
System.out.println(a);  
}  
}
```

**JAVA124.java**

```
class JAVA124{  
public static void main (String args []){  
int a=0x_1_A_F;  
System.out.println(a);  
}  
}
```

**JAVA125.java**

```
class JAVA125{  
public static void main (String args []){  
double d = 1_2_3.4_5_6;  
System.out.println(d);  
}  
}
```

**JAVA126.java**

```
class JAVA126{  
public static void main (String args []){  
double d = 123_.456;  
double d1 = 123_.456;  
System.out.println(d);  
System.out.println(d1);  
}  
}
```

**Program output /error and details:-****Boolean Literals**

| Prog No. | Output            | error and details                                                                                                                                                              |
|----------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JAVA31   | true<br>false     | Here, Data type is boolean and literals is also boolean type like true and false.                                                                                              |
| JAVA32   | Compilation error | [Error: Incompatible type] In boolean type , We can't store integer value.                                                                                                     |
| JAVA33   | Compilation error | [Error: cannot find symbols True] because True is not a boolean type.                                                                                                          |
| JAVA34   | false             | 1 <sup>st</sup> variable declare as boolean type and variable name is True value is false. Than 2 <sup>nd</sup> statement value of b is assign as True so it will print false. |

**Character Literals**

| Prog No. | Output            | error and details                                                               |
|----------|-------------------|---------------------------------------------------------------------------------|
| JAVA35   | a<br>J<br>5<br>*  | Character Literal is a single character enclosed within single quotation marks. |
| JAVA36   | Compilation error | [Error: Unclosed character Literal] Empty as character Literal not allowed.     |
| JAVA37   | [Blank]           | One space literal is valid                                                      |
| JAVA38   | [Blank]           | [Error: Invalid character constant] Two Space Literals not allowed.             |
| JAVA39   | [Blank]           | One Tab is valid Literal.                                                       |
| JAVA40   | Compilation error | [Error: Invalid character constant] Two character Literals not allowed.         |
| JAVA41   | Compilation error | [Error: Invalid character constant] '\' must be flowed by one symbol.           |
| JAVA42   | Compilation error | [Error: Unclosed character Literal] '+' and '+' is not allowed.                 |

**Escape Sequence**

| Prog No. | Output            | error and details                                                         |
|----------|-------------------|---------------------------------------------------------------------------|
| JAVA43   | '                 | To print ' you have to represent with Escape Seq.                         |
| JAVA44   | \                 | same ""                                                                   |
| JAVA45   | Compilation error | [Error: Invalid escape sequence (valid ones are \b \t \n \f \r \" \' \\)] |
| JAVA46   | Compilation error | Same"                                                                     |
| JAVA47   | Compilation error | Same"                                                                     |
| JAVA48   | "                 | It is valid Escape Seq.                                                   |
| JAVA49   | "                 | It is valid Escape Seq.                                                   |
| JAVA50   | Compilation error | [Error: expected ;]                                                       |
| JAVA51   | Welcome to " ABC" | If you want to print " value then use \" only                             |
| JAVA52   | Compilation error | [Error: Invalid escape sequence]                                          |

## ASCII Character Sets

| Prog No. | Output                       | error and details                                                                              |
|----------|------------------------------|------------------------------------------------------------------------------------------------|
| JAVA53   | 1                            | '1' is a valid character.                                                                      |
| JAVA54   | Some Symbol                  | It will print some symbol those followed by ASCII 1                                            |
| JAVA55   | 1<br>59                      | 1 <sup>st</sup> time char is display like 1 and then ASCII value of 1 is add with and display. |
| JAVA56   | 3<br>99                      | 1 <sup>st</sup> integer will add then ASCII value of 1 and 2 will be add.                      |
| JAVA57   | A<br>o<br>a                  | ASCII Value can be assigned to a char type variable.                                           |
| JAVA58   | ?<br>?                       | After rang of ASCII value it will be print ? mark<br>? mark is also a character                |
| JAVA59   | blank space<br>?             | ASCII O having blank space value.<br>After rang of ASCII value ? will be printed               |
| JAVA60   | Compilation error            | [Error: Possible lose of Precision] because value is out of range.                             |
| JAVA61   | (A+B) <sup>2</sup><br>10/2=5 | According to ASCII value it will print corresponding symbols.                                  |

## UNICODE Character

| Prog No. | Output               | error and details                                                                      |
|----------|----------------------|----------------------------------------------------------------------------------------|
| JAVA62   | ab<br>AB             | According to Unicode value it will print corresponding character.                      |
| JAVA63   | 97<br>98<br>65<br>66 | due to int data type it will print ASCII of corresponding character.                   |
| JAVA64   | 7<br>55              | Due to int data type it will print corresponding integer value.                        |
| JAVA65   | 79                   | Due to int data type it will print corresponding integer value.                        |
| JAVA66   | Compilation error    | [Error: Unclosed char literals] character cannot store two character.                  |
| JAVA67   | 99                   | Value of \u0061 is a so it is valid and print a value.                                 |
| JAVA68   | Compilation error    | [Error: Variable must be initialise] \u0061 is equivalent to 'a' and data type is int. |
| JAVA69   | 99                   | Value of \u0061 is a so it is valid and print a value.                                 |
| JAVA70   | 123                  | Source code is also written in UNICODE representation.                                 |

**Octal Value as chat type**

| Prog No. | Output            | error and details                                                 |
|----------|-------------------|-------------------------------------------------------------------|
| JAVA71   | A<br>C            | It is valid octal value.                                          |
| JAVA72   | Compilation error | [Error: Invalid Character] Octal value must have single quotation |
| JAVA73   | 65                | It is valid octal value.                                          |
| JAVA74   | Compilation error | [Error: Invalid Character] Octal value must have single quotation |
| JAVA75   | Compilation error | [Error: Unclosed char literals] Range up to 377                   |

**String Literal**

| Prog No. | Output                                                      | error and details                                                                                          |
|----------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| JAVA76   | Run Time error                                              | {Error: Null Pointer Exception} If you can refer any members with NULL String then it will give exception. |
| JAVA77   | empty<br>len : 0                                            | Empty String Literals is allowed.                                                                          |
| JAVA78   | AB12@\$ab<br>len : 8                                        | You can pass any type of parameter as String.                                                              |
| JAVA79   | ASCII of A is 65<br>UNICODE of A is 65<br>UC of A is \u0041 | You can use escape sequence to represent UNICODE/Octal value as it is in String literals.                  |
| JAVA80   | D:<br>ew est ranch<br>D:\new\test\branch                    | Same""                                                                                                     |
| JAVA81   | Compilation error                                           | [Error: Invalid escape sequence] you have to use \\ as escape sequence to print \.                         |
| JAVA82   | D:\java\core                                                | You can use escape sequence to represent UNICODE/Octal value as it is in String literals.                  |
| JAVA83   | Octal of 65 is A<br>Octal of 65 is \101                     | same""                                                                                                     |

**Decimal literals**

| Prog No. | Output               | error and details                                                                          |
|----------|----------------------|--------------------------------------------------------------------------------------------|
| JAVA84   | 65<br>1234<br>123431 | Integer Literals can be assigned to one of the integral data type- byte, short, int, long. |
| JAVA85   | Compilation error    | [Error: Out of Range] maximum rang of byte is 127                                          |
| JAVA86   | Compilation error    | [Error: Out of Range] maximum rang of short is 32767                                       |
| JAVA87   | Compilation error    | [Error: Out of Range] maximum rang of int is 214748364                                     |
| JAVA88   | Compilation error    | [Error: integer no too large] use L/l as suffix.                                           |
| JAVA89   | 2147483648           | It is valid literal for long type.                                                         |
| JAVA90   | Compilation error    | [Error: integer no too large] use L/l as suffix.                                           |
| JAVA91   | 9980979999           | It is valid literal for long type.                                                         |
| JAVA92   | Compilation error    | [Error: The literal 098 of type int is out of range] we cannot use 9 and 8 digit in octal  |

**Octal and Hexadecimal literals**

| Prog No. | Output            | error and details                                                                  |
|----------|-------------------|------------------------------------------------------------------------------------|
| JAVA93   | 1010<br>65        | 1 <sup>st</sup> is decimal and 2 <sup>nd</sup> is octal because value start with 0 |
| JAVA94   | 10<br>97<br>64206 | Hexadecimal Literals must start with 0X /0x. So that it is valid literals.         |
| JAVA95   | 64206             | same""                                                                             |

**Binary literals**

| Prog No. | Output                | error and details                                                                                                            |
|----------|-----------------------|------------------------------------------------------------------------------------------------------------------------------|
| JAVA96   | 101<br>5              | 1 <sup>st</sup> value is int type and 2 <sup>nd</sup> value is binary type.                                                  |
| JAVA97   | Compilation error     | [Error: expected ;] Only 0 and 1 is allowed to form binary literals.                                                         |
| JAVA98   | 123<br>65<br>257<br>5 | 1 <sup>st</sup> value is decimal, 2 <sup>nd</sup> value is octal, 3 <sup>rd</sup> is Hexadecimal, 4 <sup>th</sup> is Binary. |

**Floating Point Literals**

| Prog No. | Output             | error and details                                                                             |
|----------|--------------------|-----------------------------------------------------------------------------------------------|
| JAVA99   | 1234.56<br>123.456 | Value of d1 and d2 are double type.                                                           |
| JAVA100  | Compilation error  | [Error: possible loss of precision] required float and found double You must use f as suffix. |
| JAVA101  | 1234.56<br>123.456 | It is valid float point.                                                                      |

**Exponent Notation**

| Prog No. | Output                          | error and details                           |
|----------|---------------------------------|---------------------------------------------|
| JAVA102  | 12987.63<br>1287.63<br>129.8763 | It is valid Exponent notation.              |
| JAVA103  | 63.0                            | Value is octal type.                        |
| JAVA104  | 77.0                            | Value is decimal type.                      |
| JAVA105  | 169.0                           | Value is Hexadecimal type.                  |
| JAVA106  | Compilation error               | [Error: Invalid hexadecimal literal number] |

**Hexadecimal floating point**

| Prog No. | Output               | Error and details                                                  |
|----------|----------------------|--------------------------------------------------------------------|
| JAVA107  | 169.0625<br>169.0    | It is valid Hexadecimal notation                                   |
| JAVA108  | 10.0<br>20.0<br>40.0 | same''                                                             |
| JAVA109  | 60.0                 | Value is converted according to formula.                           |
| JAVA110  | 31.25                | same'''                                                            |
| JAVA111  | Compilation error    | [Error: Type mismatch: cannot convert from double to float         |
| JAVA112  | 11.0                 | Value is converted according to formula                            |
| JAVA113  | Compilation error    | [Error: Expected ;] You can't use decimal point with binary digit. |

**Underscore**

| Prog No. | Output            | error and details                                                                                                                       |
|----------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| JAVA114  | Compilation error | [Error: Expected ;] You can't to use comma(,) to represent unit of the values.                                                          |
| JAVA115  | 24452452          | You have to use Underscore ( _ ) to represent unit of the values.                                                                       |
| JAVA116  | 9898<br>9898      | When UNDERSCORE is used before first digit then it will considered as variable.                                                         |
| JAVA117  | Compilation error | [Error: cannot found symbols _ ] You can't use UNDERSCORE before the first digit.                                                       |
| JAVA118  | Compilation error | [Error: illegal underscore] You can't use UNDERSCORE after the last digit.                                                              |
| JAVA119  | 63                | It is octal value.                                                                                                                      |
| JAVA120  | 21                | It is binary value.                                                                                                                     |
| JAVA121  | Compilation error | [Error: illegal underscore] You can't use UNDERSCORE after the B.                                                                       |
| JAVA122  | 8981              | You can use more than one UNDERSCORE.                                                                                                   |
| JAVA123  | 431               | It is Hexadecimal value.                                                                                                                |
| JAVA124  | Compilation error | [Error: illegal underscore] You can't use UNDERSCORE after the X.                                                                       |
| JAVA125  | 123.456           | You can use UNDERSCORE with floating point literals.                                                                                    |
| JAVA126  | Compilation error | [Error: illegal underscore;] You can use UNDERSCORE with floating point literals but not allowed just before of after the decimal point |